

SYNAPSE

An AI-Native Workflow Orchestrator

Author: Abdallah Elrizz

Date: May 22, 2026

Status: Concept — Pre-MVP

This document captures the complete product idea for a new workflow orchestration platform built from the ground up for AI-first workloads. It covers the vision, the market gap, the architecture, the feature set, the go-to-market strategy, and the technical implementation plan. It is written to be a living reference — start here every time you return to this idea.

PART 1 — VISION

1.1 The Problem

Zapier, n8n, and Make were built for a world of linear automations — trigger app A, send data to app B, post result to app C. That world still exists. But AI workloads are fundamentally different. They require parallel execution, dynamic routing, memory, embeddings, vector search, LLM calls with context, and agents that loop, reflect, and correct themselves. None of the existing tools were designed for this. They bolt AI on top of sequential pipelines and call it done.

The result is what builders hit every day: Code nodes that time out when AI agents are present. No true parallel execution. No native memory layer. No concept of session context. Workarounds stacked on workarounds. The tools that do handle parallelism and AI natively (Temporal, Prefect, Inngest) are code-only, developer-facing, and inaccessible to the vast majority of builders who need visual tools.

1.2 The Insight

This product was born out of five days of building Cortex Brain — a biologically-inspired AI cognitive architecture in n8n. Every architectural pain point hit during that build is a product requirement for Synapse. The founder is not guessing at user problems. The founder lived them.

- n8n cannot run branches in parallel — it serializes everything, even when branches are fully independent
- Code nodes time out when AI Agent nodes are present in the same workflow — a known unfixed bug
- There is no native memory layer — every conversation starts cold unless you wire it yourself
- Context management between LLM calls is manual and fragile
- There is no concept of confidence-gated routing, agent escalation, or dynamic tool selection
- Sleep cycles, consolidation, and background processing require complex workarounds
- Debugging an AI workflow is nearly impossible — no visibility into what the LLM actually did

1.3 The Vision

Synapse is a workflow orchestration platform where AI is a first-class citizen — not a plugin, not a node type, not an afterthought. It is built from the ground up assuming that the most important workflows a builder will run involve LLMs, agents, memory, embeddings, and dynamic routing. Everything else is secondary.

Visual + accessible + true parallelism + AI-native + memory-aware. Nobody has built this yet.

1.4 The Name

Synapse. A synapse is the junction between two neurons — the point where signals pass, where learning happens, where the brain actually connects things together. It is the right metaphor for a tool that connects AI components into thinking systems. Short, technical, memorable, and biologically honest.

PART 2 — MARKET

2.1 Competitive Landscape

Tool	Parallel Execution	AI-Native	Visual	Memory-Aware	Open Source
Zapier	No	No	Yes	No	No
n8n	No	Partial	Yes	No	Yes
Make	No	No	Yes	No	No
Temporal	Yes	No	No	No	Yes
Inngest	Yes	No	No	No	Yes
Prefect	Yes	No	No	No	Yes
Windmill	Yes	Partial	Partial	No	Yes
Synapse	Yes	Yes	Yes	Yes	Yes

2.2 The White Space

The intersection of visual + accessible + parallel + AI-native + memory-aware is completely unoccupied. Every tool that does parallelism well is code-only. Every tool that is visual is sequential. Synapse sits exactly in the gap.

2.3 Target Users

- **AI Engineers** — building agents, pipelines, and cognitive architectures. Currently using n8n or raw code. Hitting ceilings daily.
- **Technical Founders** — need automation infrastructure without a DevOps team. Want visual tools that can handle real AI workloads.
- **Automation Builders** — currently on Zapier or Make, hitting limits as they try to incorporate AI into their workflows.
- **Researchers and Tinkerers** — building experimental AI systems. Need a playground that doesn't fight them.

PART 3 — ARCHITECTURE

3.1 Core Philosophy

Synapse's architecture is built around three non-negotiable principles: true parallelism at the execution engine level, AI as a first-class node type, and memory as a platform primitive rather than a user-wired feature.

3.2 Execution Engine

The execution engine is the heart of Synapse and its primary differentiator. It is built in Node.js using native `async/await` and `Promise.all()` for parallel branch execution. When the engine detects that two branches of a workflow are independent (no shared data dependency), it fires them simultaneously and waits for both before continuing.

Execution model:

- Each workflow is a directed acyclic graph (DAG) of nodes
- The engine performs a dependency analysis pass before execution
- Independent subgraphs are executed with `Promise.all()` — true parallelism
- Dependent nodes wait only for their direct inputs — no unnecessary blocking
- Every node execution is logged with timing, input, output, and error state
- Failed nodes can retry with configurable backoff without restarting the workflow

3.3 The Canvas (Visual Editor)

Built on React Flow — a production-grade node canvas library. The canvas is the interface through which users wire nodes together. It supports drag-and-drop node placement, connection drawing, node configuration panels, zoom and pan, minimap, and real-time execution visualization (nodes light up as they execute).

Canvas features:

- Drag nodes from a sidebar panel onto the canvas
- Draw connections between node output ports and input ports
- Click any node to open its configuration panel on the right
- During execution: nodes show status (pending / running / done / error) in real time
- After execution: click any node to inspect its input and output data
- Parallel branches visually highlighted during execution

3.4 Node Types

Core Nodes (built-in from day one):

Node	Purpose
-------------	----------------

Trigger: Webhook	Receive HTTP requests — entry point for external calls
Trigger: Scheduler	Cron-based scheduling — run workflows on a timer
Trigger: Manual	Run workflow manually from the canvas
HTTP Request	Call any REST API — covers 90% of integrations out of the box
Code (JS)	Run arbitrary JavaScript — full Node.js environment
Code (Python)	Run arbitrary Python — full stdlib + pip packages
IF / Switch	Conditional routing — branch based on data values
Merge / Join	Wait for parallel branches — converge multiple inputs
Set / Transform	Shape, rename, or compute data fields
Loop	Iterate over arrays — fire child nodes per item
Delay	Wait N seconds/minutes before continuing
Webhook Response	Send HTTP response back to the caller

AI-Native Nodes (the differentiator):

Node	Purpose
LLM Call	Call any model via OpenRouter or direct API. Configurable system prompt, temperature, max tokens.
LLM Agent	Autonomous agent with tool access. Loops until task complete or max iterations reached.
Embed	Generate vector embeddings via OpenAI text-embedding-3-small or other providers.
Vector Search	Query a vector database (Supabase pgvector, Pinecone, Weaviate) by similarity.
Reranker	LLM-based semantic reranking of retrieved candidates. Eliminates noise from vector search.
Memory Read	Read from the session memory layer — retrieve conversation summary, past interactions.
Memory Write	Write to the session memory layer — save interaction, update summary.
Confidence Gate	Route based on LLM-reported confidence score. High confidence goes one way, low another.
Parallel AI	Fire N LLM calls simultaneously with Promise.all(). Collect all results before continuing.
Stream Response	Stream LLM output token by token back to the user. No waiting for full completion.

3.5 Memory as a Platform Primitive

Memory is not something users wire themselves in Synapse. It is a platform-level feature. Every workflow has access to a built-in memory layer with zero configuration. This is the single biggest gap in every existing tool.

- **Session Memory** — conversation context within a single session. Automatically maintained. Accessible via Memory Read/Write nodes.
- **Semantic Memory** — long-term learned patterns and rules. Persists across sessions. Queryable by vector similarity.
- **Episodic Memory** — log of past interactions. Queryable by recency or content. Feeds background consolidation.
- **Working Memory** — temporary computation space within a single workflow execution. Cleared on completion.

3.6 Credential Management

Credentials in Synapse are workspace-level resources — created once, reused across all workflows. They are stored encrypted at rest using AES-256. They are never exposed in logs or execution traces. Node configuration panels show only credential names, never values.

- API Key credentials — name + secret key. Used by HTTP Request, LLM Call, Embed nodes.
- OAuth2 credentials — client ID + secret + token endpoint. Auto-refresh on expiry.
- Basic Auth credentials — username + password.
- Database credentials — host, port, database, username, password. Used by DB Query node.
- Environment variable injection — credentials can be referenced in Code nodes as `process.env.CRED_NAME`.
- Credential scoping — credentials can be restricted to specific workflows or shared workspace-wide.
- Audit log — every credential access is logged with timestamp, workflow ID, node ID, and user.

PART 4 — FEATURES

4.1 Execution Logs

Every workflow execution is fully logged. Logs are the primary debugging surface — not an afterthought. Every node in every execution stores:

- Start timestamp and end timestamp
- Duration in milliseconds
- Input data (full JSON, inspectable in the UI)
- Output data (full JSON, inspectable in the UI)
- Status: pending / running / success / error / skipped
- Error message and stack trace if failed
- Retry count if the node was retried
- For LLM nodes: the full prompt sent, the full response received, token counts, model used, cost
- For HTTP nodes: URL, method, request headers and body, response status, response headers and body
- For parallel branches: which branches ran simultaneously and their individual timings

Logs are searchable, filterable by status and date, and retained for 30 days on free tier and indefinitely on paid tiers. The execution timeline view shows a Gantt-style chart of node execution order and parallelism.

4.2 Real-Time Execution Visualization

While a workflow is running, the canvas updates in real time. Nodes pulse when they are actively executing. Parallel branches visually execute simultaneously. Completed nodes show a green checkmark. Failed nodes show a red X. The user can click any completed node mid-execution to inspect its data without stopping the workflow.

4.3 Error Handling

- **Retry on failure** — configurable per node. Exponential backoff. Max retry count.
- **Error branch** — every node has an optional error output port. Wire it to handle errors gracefully.
- **Continue on fail** — node failure doesn't stop the workflow. Execution continues with error data.
- **Global error handler** — a special workflow triggered whenever any workflow in the workspace fails.
- **Dead letter queue** — failed executions are queued for manual review and rerun.
- **Alerting** — send a notification (email, Slack, webhook) when a workflow fails.

4.4 Workflow Versioning

Every save creates a new version. Versions are stored indefinitely. Users can view the diff between any two versions, roll back to any prior version, and deploy a specific version to production while keeping a different version in development.

4.5 Environment Management

- Every workspace has Development and Production environments by default
- Credentials, variables, and webhook URLs are environment-scoped
- Promote a workflow from dev to production with one click
- Environment variables (API keys, config values) stored separately per environment
- Test executions always run in development — production only runs on live triggers

4.6 Subworkflows

Any workflow can be called as a subworkflow from another workflow — synchronously or asynchronously. Synchronous: the parent waits for the child to complete and receives its output. Asynchronous: the parent fires the child and continues immediately — fire-and-forget or with a callback webhook.

4.7 Template Library

A public library of workflow templates. Users can publish their workflows as templates. Templates are searchable by category, node type, and use case. One click to clone a template into your workspace and configure credentials. AI-native templates are first-class — RAG pipeline, memory agent, sleep cycle, dual-process reasoning.

4.8 AI Workflow Assistant

A chat interface in the sidebar that can build, modify, and explain workflows in natural language. Tell it what you want to automate. It generates the workflow on the canvas. Ask it why a node is failing. It reads the execution log and explains. Ask it to optimize for latency. It identifies the independent branches and rewires for parallelism.

PART 5 — INTEGRATIONS

5.1 Integration Philosophy

Synapse does not need to pre-build 400 integrations to be useful. The HTTP Request node covers any REST API out of the box — no special integration needed. Pre-built integrations are convenience wrappers that add OAuth handling, auto-populated fields, and documentation. They come after the engine is solid.

5.2 Day-One Integrations (MVP)

- OpenAI — LLM calls, embeddings, image generation
- OpenRouter — multi-model routing (DeepSeek, Claude, Gemini, GPT, Llama)
- Anthropic — direct Claude API access
- Supabase — database queries, RPC calls, vector search
- PostgreSQL — generic Postgres connection
- HTTP Request — any REST API
- Webhook — receive any HTTP payload
- Slack — send messages, read channels
- Gmail — send emails, read inbox, create drafts
- Google Sheets — read and write spreadsheet data

5.3 Phase 2 Integrations

- Notion, Airtable, Linear, Jira — project and knowledge management
- Stripe — payment events, subscription management
- Twilio — SMS and voice
- Discord, Telegram — messaging platforms
- GitHub, GitLab — code events, PR automation
- Pinecone, Weaviate, Qdrant — vector databases
- AWS S3, Google Cloud Storage — file storage
- Browserbase, Playwright — web automation and scraping
- MCP (Model Context Protocol) — dynamic tool discovery and expert routing

5.4 The HTTP Request Node as Universal Integration

Any service with a REST API is immediately usable via the HTTP Request node. Configure the URL, method, headers, and body. Attach a credential for auth. Done. This is how n8n users actually integrate with most services anyway — the pre-built nodes are convenience, not necessity. Synapse ships this on day one and builds pre-built wrappers based on user demand.

PART 6 — TECHNICAL STACK

6.1 Backend — Execution Engine

- **Runtime:** Node.js — native async/await, Promise.all() for parallelism
- **Framework:** Fastify — fast, low-overhead HTTP server
- **Queue:** BullMQ (Redis-backed) — workflow execution queue, retry logic, dead letter queue
- **Database:** PostgreSQL (Supabase) — workflow definitions, execution logs, credentials, user data
- **Vector DB:** pgvector (Supabase) — memory layer, semantic search
- **Cache:** Redis — session memory, execution state, credential cache
- **Auth:** Supabase Auth — JWT-based, OAuth2 social login, API key generation
- **Encryption:** AES-256-GCM for credential storage at rest
- **Deployment:** Railway — Docker containers, auto-deploy on push, easy scaling

6.2 Frontend — Canvas

- **Framework:** React + TypeScript
- **Canvas:** React Flow — node editor, connection drawing, zoom/pan, minimap
- **State:** Zustand — lightweight, fast workflow state management
- **Styling:** Tailwind CSS
- **Real-time:** WebSockets (Socket.io) — live execution updates to canvas
- **Code editor:** Monaco Editor (VS Code's editor) — Code node configuration
- **Data inspector:** JSON tree viewer for node input/output inspection

6.3 Database Schema (Core Tables)

workspaces — id, name, owner_id, created_at, plan

workflows — id, workspace_id, name, definition (JSONB), active, created_at, updated_at

workflow_versions — id, workflow_id, version_number, definition (JSONB), created_at, created_by

executions — id, workflow_id, status, started_at, completed_at, trigger_type, input, error

node_executions — id, execution_id, node_id, node_name, node_type, status, started_at, completed_at, duration_ms, input (JSONB), output (JSONB), error, retry_count

credentials — id, workspace_id, name, type, data (encrypted JSONB), created_at

credentials_access_log — id, credential_id, workflow_id, node_id, accessed_at, user_id

memory_patterns — id, workspace_id, content, category, confidence, hit_count, embedding vector(1536), last_invoked_at

memory_interactions — id, workspace_id, session_id, input, output, confidence, embedding vector(1536), created_at

session_summaries — session_id (PK), workspace_id, summary, turn_count, updated_at

api_keys — id, workspace_id, user_id, key_hash, name, last_used_at, created_at

users — id, email, name, created_at

workspace_members — workspace_id, user_id, role (owner|admin|editor|viewer)

6.4 API Design

Synapse exposes a full REST API so users can manage and trigger workflows programmatically. Every action available in the UI is available via API. Auth via API key or JWT bearer token.

- POST /api/v1/workflows — create workflow
- GET /api/v1/workflows — list workflows
- GET /api/v1/workflows/:id — get workflow
- PUT /api/v1/workflows/:id — update workflow
- POST /api/v1/workflows/:id/execute — trigger manual execution
- GET /api/v1/executions — list executions (filterable by workflow, status, date)
- GET /api/v1/executions/:id — get execution with full node logs
- POST /api/v1/credentials — create credential
- GET /api/v1/credentials — list credentials (names only, never values)
- POST /api/v1/webhooks/:path — trigger webhook workflow

PART 7 — SECURITY

7.1 Credential Security

- All credentials encrypted at rest with AES-256-GCM
- Encryption key stored separately from database (environment variable, never in code)
- Credentials never appear in execution logs — only credential name is logged
- Credentials never returned via API — only names and metadata
- Credential access audit log — every use is recorded
- Credential rotation — update the value without changing the name or rewiring workflows
- Row-level security on all credential tables — users can only access their own workspace credentials

7.2 Execution Isolation

- Each workflow execution runs in an isolated context — no shared state between executions
- Code nodes run in a sandboxed VM (vm2 or isolated-vm) — no filesystem access, no process spawning
- Python Code nodes run in a Docker container per execution — full isolation
- Network access in Code nodes restricted to allowlisted domains (configurable per workspace)
- Memory limits and CPU timeouts enforced per node execution

7.3 Authentication and Authorization

- JWT-based authentication with short expiry (15 min) + refresh tokens
- API keys for programmatic access — scoped to workspace, revocable
- Role-based access control: owner / admin / editor / viewer per workspace
- OAuth2 social login (Google, GitHub)
- MFA support (TOTP)
- Session management — active sessions visible, revocable

7.4 Data Privacy

- Execution logs retained for 30 days on free tier, configurable on paid tiers
- Users can delete all execution logs at any time
- Memory layer can be cleared per session, per workspace, or globally
- GDPR-compliant data deletion on account termination
- No execution data shared between workspaces or users
- Option to disable execution logging entirely for sensitive workflows

PART 8 — PRICING

8.1 Pricing Philosophy

Generous free tier to get builders in the door. Usage-based pricing on execution minutes — not on number of workflows or integrations. Memory and logging are platform primitives — included in all tiers, not add-ons.

Tier	Price	Executions/month	Active Workflows	Log Retention	Memory
Free	\$0	1,000	5	7 days	10K patterns
Builder	\$20/mo	10,000	25	30 days	100K patterns
Pro	\$79/mo	100,000	Unlimited	90 days	Unlimited
Team	\$199/mo	500,000	Unlimited	1 year	Unlimited
Enterprise	Custom	Custom	Unlimited	Custom	Custom

8.2 Self-Hosting

Synapse is open-source (core engine + canvas). Self-hosting is free forever — deploy on Railway, Render, or your own infrastructure via Docker. Cloud-hosted version is the paid product. This mirrors n8n's model exactly and is proven to work.

PART 9 — MVP PLAN

9.1 MVP Definition

The MVP proves one thing: Synapse runs AI workflows faster than n8n by executing parallel branches truly simultaneously. Everything else is secondary. The benchmark is Cortex Brain running on Synapse vs. Cortex Brain running on n8n.

9.2 MVP Scope

- Execution engine — DAG analysis, parallel execution via Promise.all(), node log storage
- Core nodes: Webhook trigger, HTTP Request, LLM Call, Embed, IF, Merge, Code (JS), Set
- Basic canvas — React Flow, drag nodes, draw connections, run workflow, view logs
- Credential management — API keys, encrypted storage, attach to nodes
- Supabase integration — built-in DB and vector search node
- Memory nodes — Memory Read, Memory Write, session summaries
- Execution log viewer — per-node timing, input/output inspection
- Webhook triggers — stable URLs, receive payloads
- Manual trigger — run from canvas with test data
- Subworkflow support — call one workflow from another (sync and async)

9.3 What is NOT in MVP

- OAuth2 integrations (use HTTP Request node instead)
- AI Workflow Assistant
- Template library
- Team/multi-user workspaces
- Billing and paid tiers
- Python Code nodes (JS only first)
- Streaming LLM responses
- Mobile canvas

9.4 Build Timeline (Estimate)

Week	Focus	Deliverable
1–2	Execution Engine	DAG parser, parallel executor, node log DB, webhook trigger
3–4	Core Nodes	HTTP Request, LLM Call, Embed, IF, Merge, Set, Code (JS)
5–6	Canvas	React Flow canvas, node sidebar, connection drawing, run button
7	Credentials + Memory	Encrypted credential storage, Memory Read/Write nodes

8	Log Viewer	Execution list, node-level log inspector, timing view
9	Cortex Port	Port Cortex Brain from n8n to Synapse, benchmark latency
10	Polish + Deploy	Bug fixes, Railway deploy, basic auth, public demo

PART 10 — GO TO MARKET

10.1 Launch Strategy

Launch as an open-source project first. Get builders using it, contributing to it, and talking about it. The cloud-hosted version launches after the open-source version has traction. This is the proven path: n8n, Temporal, Inngest all did this.

10.2 Initial Distribution Channels

- **GitHub** — open-source repo, README with benchmark vs n8n, demo GIF of parallel execution
- **LinkedIn** — founder story, building in public, Cortex Brain as the origin story
- **X/Twitter** — technical threads, benchmark results, build logs
- **Hacker News** — Show HN post on launch day
- **Reddit** — r/selfhosted, r/n8n, r/automation, r/artificial
- **Dev.to / Hashnode** — technical articles on AI-native workflow architecture
- **AI engineer communities** — Discord servers, Slack groups, forums

10.3 The Positioning Statement

Synapse is the workflow orchestrator built for AI engineers. True parallel execution. Native memory. AI-first nodes. Everything n8n should have been.

10.4 The Origin Story (Key Asset)

Cortex Brain is a powerful narrative. A biologically-inspired AI cognitive architecture, built in five days, that hit every ceiling n8n has. The founder didn't decide to build Synapse — Synapse became inevitable after building Cortex. This is a founder-market fit story that resonates with AI engineers because they recognize the pain immediately.

PART 11 — OPEN QUESTIONS

These are the questions that need answers before or during the MVP build. They are not blockers — they are decisions to make with intentionality.

- **Stack for Code node sandboxing:** isolated-vm vs vm2 vs Docker per execution. Tradeoff: security vs latency vs complexity.
- **Workflow definition format:** store as JSONB in Postgres or as YAML files in a Git repo. JSONB is simpler. Git is more powerful for versioning.
- **Self-host deployment:** Docker Compose (simple, one file) vs Helm chart (Kubernetes, complex). Start with Docker Compose.
- **Canvas library:** React Flow is the default choice. Confirm it handles 100+ node workflows without performance issues.
- **WebSocket vs polling:** real-time execution updates. WebSockets are more complex but genuinely real-time. Polling every 500ms is simpler and good enough for MVP.
- **Memory layer backend:** Supabase pgvector (already known) vs Pinecone vs Weaviate. pgvector wins for MVP — no new infrastructure.
- **Monetization timing:** build in public for 3 months before adding billing. Get users first.
- **Name finality:** Synapse is a working title. Confirm domain availability and trademark before committing.

End of document. Version 1.0, generated May 22, 2026. Author: Abdallah Elrizz.